AUDITORÍA TÉCNICA Y DE SEGURIDAD DE SOFTWARE

INFORME TÉCNICO Y FUNCIONAL EXHAUSTIVO

Auditoría de Arquitectura, Integridad Fiscal, Concurrencia de Inventarios y Seguridad RBAC

Sistema Auditado:	**Sistema POS Multi-Empresa** (Laravel 12 / PHP 8.4)	**Fecha de Evaluación:**	Noviembre 2026
Rol del Evaluador:	Arquitecto de Software Senior, Auditor de Sistemas y Especialista QA/Seguridad	**Nivel de Aislamiento:**	Multi-Tenant por Inquilino (`empresa_id`)
Alcance de Auditoría:	Controladores, Servicios, Modelos, Migraciones BD, Integridad Transaccional, Concurrencia Fiscal y RBAC		

1. RESUMEN EJECUTIVO

El sistema evaluado constituye una plataforma integral de **Punto de Venta (POS), Facturación Fiscal, Gestión Multialmacén, Control de Pasivos/Activos Financieros (CXP/CXC) y Arqueo Turno a Turno**. Su arquitectura está orientada a entornos corporativos de alta demanda y comercios minoristas/mayoristas especializados (repuestos, servicios y vehículos/motos con seriales únicos e individualizados).

La aplicación opera bajo un modelo **Multi-Empresa con segregación de datos por inquilino (`empresa_id`)**, soportando economías multimoneda con fijación de precios y compras en **USD** como moneda funcional de referencia y liquidaciones equivalentes en Bolívares (**VES**) sujetas a la tasa cambiaria oficial diaria de la empresa.

Pila Tecnológica Detectada

- **Framework Backend:** Laravel 12.x sobre PHP 8.4 (Constructor Property Promotion, Strict Typing, Enums y Pattern Matching).
- **Capa de Persistencia y Motor Relacional:** MySQL / MariaDB bajo motor InnoDB, con aislamiento transaccional `DB::transaction()` y bloqueo pesimista en fila (`lockForUpdate()`) para transacciones críticas.
- **Control de Acceso y Autorización:** `spatie/laravel-permission` integrado con middleware individualizado por ruta (`permission:modulo.accion`), FormRequests con verificación mutante y validaciones restrictivas en la capa de servicio.
- **Frontend y UI Operativa:** Arquitectura Server-Rendered basada en Blade y Bootstrap 5, acoplada a DataTables Server-Side (Yajra), Select2 y SweetAlert2 (sin dependencias de Vite ni Tailwind en las vistas del módulo operativo Sistema).
- **Generación de Documentos:** Motor DomPDF para impresión de facturas fiscales, recibos de caja, cierres X/Z y comprobantes de traslado.

2. ARQUITECTURA DE SEGURIDAD, USUARIOS Y ROLES (RBAC)

El sistema implementa un modelo de seguridad basado en roles y permisos atómicos que combina el control por inquilino con la asignación granular de privilegios.

2.1 Segregación de Empresas y Contexto de Sesión

La asignación de usuarios a empresas se gestiona a través de la tabla relacional `empresa_user` (campos `user_id`, `empresa_id`, `es_predeterminada`, `estado`). A nivel de aplicación, el método `User::empresaActiva()` resuelve la empresa de trabajo activa desde la sesión (`session('empresa_activa_id')`). **Regla de oro de seguridad:** Ninguna consulta operativa puede ejecutarse sin filtrar estrictamente por el `empresa_id` activo del contexto autenticado.

2.2 Matriz de Roles y Capacidades

Rol	Alcance de Datos	Capacidades Clave	Restricciones Estrictas
SuperAdmin	Global (Todas las empresas)	Gestión de empresas (`empresas.*`), catálogos globales de métodos de pago (`metodos_pago.*`), bypass global.	No debe operar directamente cajas registradoras de venta.
Admin	Empresa Asignada (`empresa_id`)	Gestión total de usuarios, almacenes, productos, cajas, liquidación de comisiones, arqueos y anulación. Puede facturar seleccionando caja abierta.	No puede crear empresas ni modificar métodos de pago globales.
Operador (Cajero)	Turno Propio y Sucursal	Facturación en POS (`ventas.crear`), cobros, corte X parcial, cierre ciego Z de su propio turno (`cajas.cerrar`) y abonos CXC.	Descuentos o anulaciones requieren permisos explícitos. No visualiza costos de compra.
Vendedor	Preventas y Comisiones Propias	Acceso a POS en modo preventa (`pos.preventa`), catálogo consultor de precios (`consultor.ver`), visualización de sus ventas.	**Bloqueado:** No puede facturar, cobrar, abrir turnos, ver costos ni liquidar comisiones.

2.3 Validación de Permisos a Nivel de Código

El control de acceso no descansa únicamente en la interfaz visual, sino que se ejecuta en múltiples capas de protección:

- **Protección de Rutas (`routes/admin.php`):** Cada endpoint cuenta con el middleware de permiso específico, por ejemplo `middleware('permission:ventas.anular')` o `middleware('permission:cajas.cerrar')`.
- **FormRequests:** Las clases de validación ejecutan `$this->user()->can('permiso')` en su método `authorize()`.
- **Capa de Servicio (`app/Service/**`):** Los servicios validan la lógica de negocio antes de mutar la base de datos (ej. `VentaClass` arroja `AuthorizationException` si un usuario con rol Vendedor intenta invocar la facturación directa).
- **Directivas Blade:** Las acciones críticas de mutación se encuentran encapsuladas bajo `@can('modulo.accion')`.

3. ANÁLISIS EXHAUSTIVO POR MÓDULO

3.1 Módulo de Punto de Venta (POS) y Facturación

Gestionado por `PosController`, `FacturaController` y la clase central `VentaClass`.

- **Preventa / Ventas en Espera (`ventas_en_espera`):** Permite a los asesores comerciales crear órdenes en espera guardadas en formato JSON (`datos_json`). No congela inventario ni crea transacciones de caja. Al recuperarse en caja, el `vendedor_id` original se preserva como inalterable para garantizar el reconocimiento de comisiones.
- **Control de Cajas para Facturación:** Ninguna venta puede emitirse con `caja_id` o `caja_turno_id` nulo. El Cajero solo puede usar su turno activo; el Administrador debe seleccionar una caja abierta de la empresa activa.
- **Políticas de Precios y Descuentos:** Se validan los precios contra los valores de lista en servidor (minorista o mayorista). Toda desviación o descuento mayor al 0% exige el permiso `ventas.descuentos`. Se valida estrictamente que el precio neto no sea inferior al costo unitario (`precioNetoUnitarioUsd >= costoUnitarioUsd`).

3.2 Correlativos, Concurrencia y Control Fiscal

> **Garantía de Secuencia Atómica y Cero Colisiones**
> La generación de numeración fiscal no utiliza funciones vulnerables a condiciones de carrera como `MAX(id)`. En su lugar, utiliza la tabla `secuencias` administrada por el modelo `Secuencia` mediante bloqueos transaccionales pesimistas.

El método `Secuencia::siguiente(int $empresaId, string $tipo): int` bloquea con `lockForUpdate()` la fila correspondiente en la tabla `secuencias`, evalúa el valor máximo existente en la tabla física de origen (mediante regex sobre `codigo` y `numero_control`), incrementa en +1 y guarda el nuevo valor antes del commit.

- **Código de Factura:** Formato `VEN-00001` (relleno a 5 dígitos).
- **Número de Control Fiscal:** Formato `00-00000001` (prefijo de 2 dígitos y 8 dígitos correlativos consecutivos), adaptado a la normativa fiscal sobre formularios continuos.
- **Unicidad Garantizada en BD:** La tabla `ventas` posee restricciones físicas compuestas: `UNIQUE KEY (empresa_id, codigo)` y `UNIQUE KEY (empresa_id, numero_control)`.

3.3 Gestión de Anulaciones y Devoluciones de Venta

El sistema prohíbe el borrado físico de comprobantes. Ante una anulación (`VentaClass::anularVenta`):

1. Se requiere motivo justificado (≥ 5 caracteres) y turno de caja abierto actual.
2. La factura se actualiza a `estado = 'anulada'`, registrando `anulada_por_id`, `anulada_at` y `motivo_anulacion`.
3. Se restituye el inventario no devuelto a través del Kardex con `tipo_movimiento = 'anulacion_venta'`. Seriales y motos retornan a estado `'disponible'`.
4. Si existía cuenta por cobrar (`cuentas_por_cobrar`), se anula y sus saldos se llevan a 0.
5. El dinero efectivamente cobrado (menos devoluciones previas) se reembolsa registrando un `egreso` en `caja_movimientos` dentro del **turno de caja abierto actual** del usuario que anula, preservando la inmutabilidad de turnos históricos cerrados.
6. La comisión del vendedor se anula (`comision_monto_usd = 0`).

3.4 Gestión de Inventarios y Kardex Multialmacén

El módulo (`KardexClass`, `KardexController`) opera bajo el principio de **Kardex Perpetuo**.

- **Tabla `producto_stock_almacenes`:** Registra las existencias por producto y almacén. Antes de cualquier mutación, se bloquea la fila mediante `lockForUpdate()`.
- **Regla Estricta contra Stock Negativo:** Si `$stockAnterior $cantidad` en movimientos de salida, el sistema lanza una excepción inmediata y aborta la transacción completa.
- **Traslados entre Almacenes:** Gestionados por `procesarTraslado`. Genera código `TRA-XXXXX` y dos asientos en Kardex: `traslado_salida` en el almacén emisor y `traslado_entrada` en el almacén receptor, actualizando la ubicación física de seriales.
- **Ajustes de Inventario:** Genera código `AJU-XXXXX` para ingresos o egresos por merma, rotura o descuadre físico.

3.5 Cuentas por Pagar (CXP) y Compras

Administrado por `RecepcionClass` y `CuentaPorPagarClass`.

- **Ingreso de Mercancía (`recepciones`):** Aumenta el stock en el almacén indicado, actualiza el precio de costo del producto y la relación con el proveedor (`producto_proveedores`). Si la compra es a crédito, se genera la deuda en `cuentas_por_pagar`.
- **Liquidación FIFO de Deudas:** La función `abonarGeneralDeuda` permite aplicar pagos globales que se distribuyen automáticamente liquidando las facturas más antiguas (orden ascendente por `fecha_emision`).

3.6 Cajas, Turnos y Arqueos (Corte X / Cierre Z)

Controlado por `CajaClass`. Impone la regla: **máximo 1 turno abierto por caja y 1 turno abierto por usuario**.

- **Fórmula de Efectivo Esperado:** `Apertura + Pagos_Efectivo_Ventas + Abonos_CXC_Efectivo + Ingresos_Menores - Vueltos_Entregados - Reembolsos_Devolución - Egresos_Menores`.
- **Corte X:** Arqueo parcial en tiempo real que no modifica el estado del turno.
- **Cierre Z:** Arqueo definitivo. Para el cajero es un **cierre a ciegas** (debe declarar el conteo físico de USD y Bs sin conocer el monto esperado por el sistema). Al guardar, se calculan y registran las diferencias de caja (sobrante o faltante).

4. FLUJOS CRÍTICOS DEL SISTEMA

4.1 Flujo de Facturación, Correlativos y Control Fiscal

Paso 1: Validación Previa y Turno de Caja POSCONTROLLER

El servidor valida permisos (`ventas.crear`). Verifica la existencia de un turno de caja abierto (`caja_turnos.estado = 'abierta'`). Si es cajero, restringe a su turno activo; si es admin, exige selección obligatoria de caja abierta.

Paso 2: Inicio de Transacción y Control de Stock DB::TRANSACTION

Bloqueo pesimista sobre `producto_stock_almacenes`, seriales y motos con `lockForUpdate()`. Se verifica que `cantidad_actual >= cantidad_solicitada`. Si falta stock, ejecuta `ROLLBACK`.

Paso 3: Generación Atómica de Correlativo y Control SECUENCIA

Invocación de `Secuencia::siguiente(empresa_id, 'venta')` y `Secuencia::siguiente(empresa_id, 'numero_control')`. Se obtiene el siguiente número secuencial bajo bloqueo en la tabla `secuencias`. Se formatea como `VEN-XXXXX` y `00-XXXXXXXX`.

Paso 4: Persistencia de Venta, Kardex y Pagos INSERT

Se inserta la cabecera en `ventas` y renglones en `venta_detalles`. Se crea asiento en `kardex` (`tipo_movimiento = 'salida_venta'`). Se descuenta el stock en `producto_stock_almacenes`. Se insertan los métodos de pago en `venta_pagos`.

Paso 5: Vuelto, CXC y Consumo de Preventa FINALIZACIÓN

Si hubo vuelto en efectivo, se genera un `egreso` en `caja_movimientos`. Si hay saldo pendiente, se crea el crédito en `cuentas_por_cobrar`. Si provenía de una preventa en espera, se elimina de `ventas_en_espera`. Se confirma con `COMMIT`.

4.2 Flujo de Anulación de Facturas y Afectación de Stock

Paso 1: Solicitud y Validación de Seguridad FACTURACONTROLLER

Verificación del permiso `ventas.anular`. Exige motivo obligatorio (≥ 5 caracteres) y la existencia de un turno de caja abierto en el usuario actual para registrar el reembolso.

Paso 2: Bloqueo Pesimista del Documento LOCKFORUPDATE

Se bloquea la venta: `SELECT * FROM ventas WHERE id = ? FOR UPDATE`. Se valida que el estado sea `'completada'` o `'devuelta_parcial'`. Facturas ya anuladas son rechazadas.

Paso 3: Restitución de Stock Físico en Kardex KARDEXCLASS

Para cada renglón, se calcula la cantidad neta no devuelta previamente. Se genera movimiento en `kardex` (`tipo_movimiento = 'anulacion_venta'`) y se incrementa `cantidad_actual` en

`producto_stock_almacenes`. Motos y seriales vuelven a `'disponible'`.

Paso 4: Anulación de Deuda y Reembolso en Caja CAJAMOVIMIENTO

Si existe registro en `cuentas_por_cobrar`, su estado cambia a `'anulada'` y saldos a 0. Se calcula el efectivo neto cobrado y se registra un movimiento de `egreso` en `caja_movimientos` sobre el **turno de caja abierto actual**.

Paso 5: Cierre de Auditoría UPDATE

La venta se actualiza: `estado = 'anulada'`, `anulada_por_id`, `anulada_at`, `motivo_anulacion`, y se neutraliza la comisión del vendedor (`comision_monto_usd = 0`). `COMMIT`.

4.3 Flujo de Cuentas por Pagar y Liquidación FIFO de Proveedores

Fase A: Origen del Pasivo (Recepción de Compras) RECEPCIONCLASS

Se registra el ingreso de mercancía en `recepciones` y `recepcion_detalles`. Se crea asiento en Kardex (`entrada_recepcion`) y se actualiza el costo en `productos`. Si la compra fue a crédito, se inserta la deuda en `cuentas_por_pagar` (`estado = 'pendiente'`).

Fase B: Pago a Factura Específica CUENTAPORPAGARCLASS

Se bloquea la cuenta específica con `lockForUpdate()`. Se crea el registro en `cuentas_por_pagar_abonos`. Se descuenta el `saldo_pendiente_usd`. Si llega a cero, el estado cambia a `'pagada'`; de lo contrario, `'parcial'`.

Fase C: Abono General FIFO a Proveedor ALGORITMO FIFO

Se consultan todas las facturas pendientes del proveedor ordenadas por fecha: `ORDER BY fecha_emision ASC FOR UPDATE`. El monto del abono se aplica consecutivamente a la factura más antigua hasta saldarla o agotar el dinero, creando registros individuales de abono por cada documento liquidado.

5. VALIDACIÓN DE INTEGRIDAD, HALLAZGOS Y CONCLUSIONES

5.1 Evaluación Crítica de la Lógica Implementada

El sistema demuestra un alto rigor en sus decisiones arquitectónicas clave:

- **Control Concurrente Real:** El uso de bloqueos pesimistas en base de datos (`lockForUpdate()`) para la generación de secuencias fiscales, reserva de seriales/motos y validación de stock resuelve de raíz los problemas habituales de sobreventa y saltos de correlativos en entornos multiusuario.
- **Trazabilidad Fiscal Inmutable:** La imposibilidad de eliminar registros de venta o compras, complementada con documentos rectificativos inversos (anulaciones y devoluciones con auditoría de usuario, fecha y motivo), satisface las exigencias de auditoría contable y tributaria.
- **Aislamiento de Turnos Históricos:** Al registrar los reembolsos de dinero en el turno activo actual del usuario que anula, se garantiza que los cierres de caja históricos permanezcan inalterables y cuadren con sus depósitos bancarios correspondientes.

5.2 Matriz de Hallazgos y Recomendaciones de Mejora

Severidad	Hallazgo Detectado	Impacto Potencial	Recomendación Técnica
MEDIA	**Abonos CXP desvinculados del arqueo de caja**	Al registrar abonos a proveedores en `CuentaPorPagarClass`, no se crea automáticamente el egreso en `caja_movimientos`. Si se paga con dinero del cajón, se produce un sobrante artificial en el turno.	Incorporar un parámetro opcional `pagar_desde_caja` que exija turno abierto e inserte el movimiento de salida en `caja_movimientos`, tal como se implementó en la liquidación de comisiones.
BAJA	**Índices únicos condicionales en migración**	La migración de secuencias verifica la existencia de duplicados antes de aplicar los índices `UNIQUE` a nivel de motor de BD.	Ejecutar un script de saneamiento de datos históricos para forzar la presencia del índice físico `UNIQUE` en el 100% de las instancias.
INFORMATIVA	**Módulo IGTF inerte**	Los campos de IGTF en la tabla `ventas` están presentes pero su cálculo está pausado según la decisión funcional D5.	Mantener aislada la lógica hasta la activación regulatoria oficial para no generar distorsiones en los libros contables.

5.3 Conclusión y Dictamen Final

> **Dictamen de Auditoría: APROBADO CON EXCELENCIA TÉCNICA**
>
> La arquitectura del Sistema POS evaluado satisface ampliamente los estándares de la industria en cuanto a resiliencia transaccional, seguridad basada en roles (RBAC), control estricto de correlativos fiscales concurrentes y control perpetuo de inventarios. Se recomienda implementar la automatización del egreso de caja en pagos a proveedores para cerrar el ciclo financiero al 100%.